SALUS SECURITY

MAY 2026

CODE SECURITY ASSESSMENT

ACH CHAIN

Overview

Project Summary

- Name: Alchemy-Chain
- Platform: EVM-compatible chains
- Language: GO, Solidity
- Audit Range: See Appendix - 1

Project Dashboard

Application Summary

Name	Alchemy-Chain
Version	v2
Type	Solidity
Dates	May 28 2025
Logs	Apr 24 2025; May 28 2025

Vulnerability Summary

Total High-Severity issues	10
Total Medium-Severity issues	4
Total Low-Severity issues	1
Total informational issues	1
Total	16

Contact

E-mail: support@salusec.io

Risk Level Description

High Risk	The issue puts a large number of users' sensitive information at risk, or is reasonably likely to lead to catastrophic impact for clients' reputations or serious financial implications for clients and users.
Medium Risk	The issue puts a subset of users' sensitive information at risk, would be detrimental to the client's reputation if exploited, or is reasonably likely to lead to a moderate financial impact.
Low Risk	The risk is relatively small and could not be exploited on a recurring basis, or is a risk that the client has indicated is low impact in view of the client's business circumstances.
Informational	The issue does not pose an immediate risk, but is relevant to security best practices or defense in depth.

Content

Introduction

1.1 About SALUS

At Salus Security, we are in the business of trust.

We are dedicated to tackling the toughest security challenges facing the industry today. By building foundational trust in technology and infrastructure through security, we help clients to lead their respective industries and unlock their full Web3 potential.

Our team of security experts employ industry-leading proof-of-concept (PoC) methodology for demonstrating smart contract vulnerabilities, coupled with advanced red teaming capabilities and a stereoscopic vulnerability detection service, to deliver comprehensive security assessments that allow clients to stay ahead of the curve.

In addition to smart contract audits and red teaming, our Rapid Detection Service for smart contracts aims to make security accessible to all. This high calibre, yet cost-efficient, security tool has been designed to support a wide range of business needs including investment due diligence, security and code quality assessments, and code optimisation.

We are reachable on Telegram (https://t.me/salusec), Twitter (https://twitter.com/salus_sec), or Email (support@salusec.io).

1.2 Audit Breakdown

The objective was to evaluate the repository for security-related issues, code quality, and adherence to specifications and best practices. Possible issues we looked for included (but are not limited to):

- Risky external calls
- Integer overflow/underflow
- Transaction-ordering dependence
- Timestamp dependence
- Access control
- Call stack limits and mishandled exceptions
- Number rounding errors
- Centralization of power
- Logical oversights and denial of service
- Business logic specification
- Code clones, functionality duplication

1.3 Disclaimer

Note that this security audit is not designed to replace functional tests required before any software release and does not give any warranties on finding all possible security issues with the given smart contract(s) or blockchain software, i.e., the evaluation result does not guarantee the nonexistence of any further findings of security issues.

Findings

2.1 Summary of Findings

ID	Title	Severity	Category	Status
1	Unrestricted Token Issuance Allows Anyone to Consume the Platform Gas Payer	High	Access Control	Resolved
2	Missing chainId in Signed Messages Enables Cross-Chain Replay	High	Cryptography	Resolved
3	The `MethodName` Is Excluded from the Signature, Enabling Cross-Method Replay	High	Cryptography	Resolved
4	The `IssueParams.nonce` and `DynamicCallParams.nonce` do not prevent signature replay	High	Cryptography	Resolved
5	The `recentCheckpoint` Is Documented as Replay Protection but Is Not Enforced	High	Business Logic	Resolved
6	Dynamic Contract Execution Lacks a Target Contract Allowlist	High	Business Logic	Resolved
7	Future Nonce Transactions Enable TxPool Queue Exhaustion DoS Against the Gas Payer	High	Denial of Service	Resolved
8	Inconsistent TxPool and ERC20 Gas Charging Logic Can Cause Invalid Transaction Flooding	High	Business Logic	Resolved
9	Blacklisting address(0) Breaks Minting and Burning	High	Business Logic	Resolved
10	Duplicate Role Entries Can Leave Revoked Accounts Authorized	High	Business Logic	Resolved
11	Excessive Genesis gasLimit Can Enable Resource Exhaustion	Medium	Business Logic	Resolved
12	Ambiguous Flat Message Encoding Enables Signature Message Collisions	Medium	Cryptography	Resolved
13	Dynamic Contract Calls Use Fixed Sleep and Long Polling, Enabling Application-Level DoS	Medium	Denial of Service	Resolved
14	Integer Arguments Can Suffer Silent Precision Loss Through float64	Medium	Numerics	Resolved
15	Hardcoded Private Keys in Test and Script Files	Low	Configuration	Resolved
16	The parseContractResult Does Not Validate Empty Return Values	Informational	Business Logic	Resolved

2.2 Notable Findings

Significant flaws that impact system confidentiality, integrity, or availability are listed below.

1. Unrestricted Token Issuance Allows Anyone to Consume the Platform Gas Payer

Severity: High	Category: Access Control

Target:
- rest/tokens/crypto/signature.go
- rest/tokens/service.go

Description

The `issueToken` flow only verifies that the recovered signer matches `MasterAuthority`. It does not verify whether the `MasterAuthority` is an approved issuer or registered platform partner.

rest/tokens/crypto/signature.go:L33–53

```
func (v *DefaultSignatureVerifier) VerifySignature(params *tokentypes.IssueParams) error
{
	if params.Signature == nil {
		return fmt.Errorf("signature cannot be empty")
	}

	// 使用动态签名验证方法
	signerAddress, err := v.VerifyDynamicSignature(params, params.Signature)
	if err != nil {
		return fmt.Errorf("signature verification failed: %w", err)
	}

	// 验证签名者是否为主权限地址
	masterAddress := common.HexToAddress(params.MasterAuthority)
	recoveredAddress := common.HexToAddress(signerAddress)
	if recoveredAddress != masterAddress {
		return fmt.Errorf("signature verification failed: signer address %s does
not match master authority address %s",
			signerAddress, masterAddress.Hex())
	}

	return nil
}
```

Because token deployment is paid by the platform `gas payer`, any external user can set themselves as `MasterAuthority`, self-sign the request, and cause the backend to deploy both the implementation contract and proxy contract on their behalf. This consumes the platform gas payer's nonce and gas balance without authorization.

Recommendation

Add an allowlist or on-chain registry for approved token issuers. After signature verification and before deployment, check that `MasterAuthority` is registered and authorized to call `create_token / issueToken`.

Return an explicit error when the issuer is not approved. Document that gas-sponsored token creation is only available to registered issuers or partners.

Status

This issue has been resolved by the team.

2. Missing chainId in Signed Messages Enables Cross-Chain Replay

Severity: High	Category: Cryptography
Target: - rest/tokens/crypto/signature.go - rest/tokens/types/request.go	

Description

The signed message generated by BuildDynamicSignatureMessage does not include the current network chainId. As a result, a signature that is valid on one network may also be valid on another network if the same request parameters are accepted.

For example, a request signed for a testnet deployment or contract call may be replayed on mainnet if the backend and contract state accept the same parameters.

Recommendation

Include chainId in every signed request domain. The backend should compare the signed chainId against eth_chainId before accepting the signature.

Prefer EIP-712 typed structured data with a domain separator that includes chainId, verifying contract / service identifier, and intended action.

Status

This issue has been resolved by the team.

3. The `MethodName` Is Excluded from the Signature, Enabling Cross-Method Replay

Severity: High	Category: Cryptography
Target: - rest/tokens/service.go - rest/tokens/crypto/signature.go	

Description

In the dynamic contract call flow, the backend builds the signature message through `extractDynamicParams`. During this process, extractDynamicParams explicitly skips both MethodName and Signature, so MethodName is not included in the signed message.

As a result, the recovered signature only proves approval over the extracted parameters, but it does not cryptographically bind the signature to the specific method the user intended to call.

If two methods accept the same argument structure, a signature intended for one method may be replayed against another method. For example, a signature over parameters such as (address, uint256) may be reused between methods like `mint(address to, uint256 amount)` and `adminBurn(address account, uint256 amount)` if the parameter values are compatible.

Recommendation

Include MethodName in the signed payload generated for dynamic contract calls.

Status

This issue has been resolved by the team.

4. The `IssueParams.nonce` and `DynamicCallParams.nonce` do not prevent signature replay

Severity: High	Category: Cryptography

Target:
- rest/tokens/service.go

Description

IssueParams.Nonce and DynamicCallParams.Nonce are present in the request structures, but they are not used as replay-protection nonces.

In the token issuance flow, the provided nonce is compared against the gas payer account's pending Ethereum nonce. If the provided nonce is lower than the pending nonce, the backend silently replaces it with the pending nonce and continues execution. This allows an old signed request to be submitted again after the gas payer nonce has advanced.

In the dynamic call flow, the nonce is only checked to ensure it is non-negative. There is no persistence layer, no used-nonce tracking, and no monotonic nonce enforcement for the signer.

rest/tokens/service.go:L263–288

```
func (tokenService *TokenService) handleNonce(ctx context.Context, ethClient
*ethclient.Client, transactor *bind.TransactOpts, privateKey *ecdsa.PrivateKey,
requestNonce *int64) error {
	sender := crypto.PubkeyToAddress(privateKey.PublicKey)
	pendingNonce, err := ethClient.PendingNonceAt(ctx, sender)
	if err != nil {
		return err
	}

	if requestNonce != nil {
		if *requestNonce < 0 {
			return errors.New("nonce must be greater than or equal to 0")
		}

		providedNonce := uint64(*requestNonce)
		useNonce := providedNonce

		// 如果提供的 nonce 过期，使用待处理的 nonce
		if providedNonce < pendingNonce {
			log.Warn("Updating expired nonce", "provided", providedNonce,
"pending", pendingNonce)
			useNonce = pendingNonce
		}

		transactor.Nonce = new(big.Int).SetUint64(useNonce)
	}

	return nil
```

```
}
```

Recommendation

Separate the user replay-protection nonce from the gas payer's transaction nonce.

Store and enforce signer-specific nonces server-side or on-chain. Reject reused, stale, or non-monotonic user nonces. Do not silently replace a signed nonce with the gas payer's pending transaction nonce.

Status

This issue has been resolved by the team.

5. The `recentCheckpoint` Is Documented as Replay Protection but Is Not Enforced

Severity: High	Category: Business Logic
Target: - rest/tokens/utils/validation.go - rest/tokens/service.go	

Description

The request body and documentation include `recentCheckpoint`, and the documentation describes nonce / checkpoint fields as replay-protection mechanisms. However, the backend only checks that RecentCheckpoint >= 0.

There is no validation against the current chain height, no time window, no persisted checkpoint state, and no marking of requests as consumed. An attacker who captures a valid historical request can replay it multiple times while the signature remains valid.

Recommendation

Define the exact semantics of recentCheckpoint. For example, require it to be within a bounded block-height window and reject requests that are too old or too far in the future.

Status

This issue has been resolved by the team.

6. Dynamic Contract Execution Lacks a Target Contract Allowlist

Severity: High	Category: Business Logic

Target:
- rest/tokens/service.go
- rest/tokens/contracts/Permission.sol

Description

The backend accepts a user-supplied token / contract address for dynamic contract calls. The validation mainly checks that the method exists in the local ABI, that the argument count matches, and that the target address has deployed code.

rest/tokens/service.go:L703–734

```
func (tokenService *TokenService) callContractMethodWithArgs(ctx context.Context,
tokenAddress string, methodName string, args []interface{}) (interface{}, error) {
	contractAddress := common.HexToAddress(tokenAddress)
	baseContract, err :=
tokenService.loadContract(tokenService.config.BaseContractPath)
	if err != nil {
		return nil, fmt.Errorf("failed to load base contract: %w", err)
	}
	contract := bind.NewBoundContract(contractAddress, baseContract.ABI, ethClient,
ethClient, ethClient)
	code, err := tokenService.waitForContractCode(ctx, ethClient, contractAddress)
	if err != nil {
		return nil, err
	}
	log.Info("Contract verification passed", "contractAddress",
contractAddress.Hex(), "codeLength", len(code))
}
```

After validation, the backend uses the platform gas payer to send the transaction. An attacker can deploy an ABI-compatible contract that returns crafted metadata or permission data, then submit calls that pass the backend's ABI and permission checks. This causes the backend to pay gas for attacker-controlled contract interactions.

Recommendation

Restrict state-changing calls to platform-registered token contracts only.

Resolve valid contract addresses from a trusted registry or allowlist instead of accepting arbitrary user-supplied contract addresses. The permission check should not rely solely on metadata returned by the target contract itself.

Status

This issue has been resolved by the team.

7. Future Nonce Transactions Enable TxPool Queue Exhaustion DoS Against the Gas Payer

Severity: High	Category: Denial of Service

Target:
- rest/tokens/service.go

Description

In the token issuance flow, the request nonce is passed into handleNonce and is eventually used as the Ethereum transaction nonce for the platform gas payer.

rest/tokens/service.go:L263–288

```
func (tokenService *TokenService) handleNonce(ctx context.Context, ethClient
*ethclient.Client, transactor *bind.TransactOpts, privateKey *ecdsa.PrivateKey,
requestNonce *int64) error {
	sender := crypto.PubkeyToAddress(privateKey.PublicKey)
	pendingNonce, err := ethClient.PendingNonceAt(ctx, sender)
	if err != nil {
		return err
	}

	if requestNonce != nil {
		if *requestNonce < 0 {
			return errors.New("nonce must be greater than or equal to 0")
		}

		providedNonce := uint64(*requestNonce)
		useNonce := providedNonce

		// 如果提供的 nonce 过期，使用待处理的 nonce
		if providedNonce < pendingNonce {
			log.Warn("Updating expired nonce", "provided", providedNonce,
"pending", pendingNonce)
			useNonce = pendingNonce
		}

		transactor.Nonce = new(big.Int).SetUint64(useNonce)
	}

	return nil
}
```

The implementation compares the user-provided nonce with the gas payer's current pendingNonce. If the provided nonce is lower than pendingNonce, the backend replaces it with pendingNonce. However, if the provided nonce is higher than pendingNonce, the backend accepts and uses the user-provided future nonce directly.

This allows an attacker to repeatedly submit token issuance requests with incrementally higher future nonces. The backend will sign and broadcast transactions from the gas payer account using those future nonces. These transactions may enter the account's queued

transaction pool as future transactions instead of being immediately executable.

By filling the gas payer account's queued transaction slots, an attacker can prevent or delay legitimate transactions from the same gas payer account. This results in a denial-of-service condition against the platform's transaction-sending capability.

Recommendation

Do not allow external users to control the gas payer's Ethereum account nonce.

Status

This issue has been resolved by the team.

8. Inconsistent TxPool and ERC20 Gas Charging Logic Can Cause Invalid Transaction Flooding

Severity: High	Category: Business Logic
Target: - core/txpool/validation.go - core/types/transaction.go - core/state/statedb.go - core/state_transition.go	

Description

When ERC20 gas mode is enabled, `StateDB.GetBalance` returns an ERC20 balance view for non-owner accounts. The txpool balance validation compares this balance against `tx.Cost()`, which is based on the transaction gas fee cap / gas price.

During execution, however, TransactionToMessage replaces msg.GasPrice with the value returned by GetContractCoefficientRPC, and buyGas charges ERC20 gas using `gasLimit * msg.GasPrice` via ExecuteAdminTransfer.

This creates a mismatch between txpool admission and execution-time charging. If the system coefficient is higher than the user-provided gas fee cap, the txpool may accept a transaction that later fails during ERC20 gas payment. Attackers can mass-submit such transactions to consume validation / block-building resources and occupy txpool slots, while failed transactions may not be charged effective execution gas.

Recommendation

Make txpool validation and execution-time gas charging use the same cost formula.

Before ERC20 buyGas, verify the user's ERC20 balance against the exact amount that ExecuteAdminTransfer will charge. Alternatively, make the txpool use a conservative ERC20 gas upper bound such as `gasLimit * max(GasFeeCap, systemCoefficientUpperBound)`.

Status

This issue has been resolved by the team.

9. Blacklisting address(0) Breaks Minting and Burning

Severity: High	Category: Business Logic

Target:
- rest/tokens/contracts/ERC20BaseUpgradeable.sol

Description

Accounts with BLACK_ROLE can call addToBlacklist. The function does not reject address(0).

rest/tokens/contracts/ERC20BaseUpgradeable.sol:L263–269

```
function addToBlacklist(address account) external onlyRole(BLACK_ROLE) {
    if (!isBlacklisted(account)) {
        _blacklistArray.push(account);
        emit Blacklisted(account);
    }
}
```

OpenZeppelin ERC20 minting internally calls _update with from == address(0), and burning calls _update with to == address(0). This contract overrides _update and checks both from and to against the blacklist without excluding the zero address.

If address(0) is blacklisted, mint and adminBurn will revert.

Recommendation

Reject account == address(0) in addToBlacklist.

Status

This issue has been resolved by the team.

10. Duplicate Role Entries Can Leave Revoked Accounts Authorized

Severity: High	Category: Business Logic
Target: - rest/tokens/contracts/ERC20BaseUpgradeable.sol - rest/tokens/contracts/Permission.sol	

Description

The contract uses OpenZeppelin AccessControl for actual role assignment, but also maintains parallel role member arrays such as _mintRoleMembers for metadata and permission lookup.

OpenZeppelin prevents duplicate role grants internally, but _addToRoleArray unconditionally pushes the account into the corresponding array. If the same account is granted the same role multiple times, duplicate entries are created in the array.

When revokeAuthority is called, _revokeRole removes the role from OpenZeppelin's role mapping, but _removeFromRoleArray removes only the first matching array entry and then breaks. Duplicate entries may remain.

Permission.getPermission checks the arrays returned by getTokenMetadata() instead of calling hasRole. Therefore, a revoked account may still be treated as authorized by the backend permission contract.

Recommendation

Use a mapping type instead of an array.

Status

This issue has been resolved by the team.

11. Excessive Genesis gasLimit Can Enable Resource Exhaustion

Severity: Medium	Category: Business Logic
Target: - genesis.json	

Description

The genesis configuration sets gasLimit to 0x1fffffffffffff, which is extremely high.

Block gas limits are intended to cap the amount of computation that can be included in a block. If the configured limit is far above the practical capacity of validator nodes, malicious or poorly bounded transactions may cause excessive CPU, memory, or execution time usage. Oversized blocks can slow synchronization, increase block processing time, and degrade network stability.

Recommendation

Set gasLimit to a realistic operational value based on expected throughput and validator hardware capacity.

Ensure the consensus and block production logic includes safe gas-limit adjustment rules and monitoring for abnormal block resource consumption.

Status

This issue has been resolved by the team.

12. Ambiguous Flat Message Encoding Enables Signature Message Collisions

Severity: Medium	Category: Cryptography

Target:
- rest/tokens/crypto/signature.go
- rest/tokens/utils/validation.go

Description

The `buildSortedMessage` sorts parameter keys and concatenates only the values using commas. It does not include field names, type tags, length prefixes, or structured boundaries. For array values, elements are expanded and appended into the same flat list.

rest/tokens/crypto/signature.go:L192–236

```
func buildSortedMessage(params map[string]interface{}) string {
	// 获取所有键并排序
	keys := make([]string, 0, len(params))
	for key := range params {
		keys = append(keys, key)
	}
	sort.Strings(keys) // a-z排序

	// 按排序后的键构建消息字符串
	var messageParts []string
	for _, key := range keys {
		value := params[key]

		// 处理数组类型参数(如 methodArgs)
		valueType := reflect.TypeOf(value)
		if valueType.Kind() == reflect.Slice || valueType.Kind() == reflect.Array
{
			sliceValue := reflect.ValueOf(value)
			// 将数组中的每个元素作为独立的签名字段
			var arrayElements []string
			for i := 0; i < sliceValue.Len(); i++ {
				element := sliceValue.Index(i).Interface()
				if element != nil {
					arrayElements = append(arrayElements,
fmt.Sprintf("%v", element))
				}
			}
			messageParts = append(messageParts, arrayElements...)
		} else {
			formattedValue := fmt.Sprintf("%v", value)
			messageParts = append(messageParts, formattedValue)
		}
	}

	return strings.Join(messageParts, ",")
}
```

This encoding is ambiguous. Different semantic inputs can produce the same message string. For example, ["A", "B"] and ["A,B", ""] can both collapse into the same comma-separated representation, causing identical hashes for different request meanings.

Recommendation

Replace flat comma-joined encoding with EIP-712 typed structured data.

If EIP-712 is not used, adopt a canonical encoding that includes field names, types, value lengths, and unambiguous nesting, such as canonical JSON with strict schema rules, protobuf, or RLP-like encoding.

Status

This issue has been resolved by the team.

13. Dynamic Contract Calls Use Fixed Sleep and Long Polling, Enabling Application-Level DoS

Severity: Medium	Category: Denial of Service
Target: - rest/tokens/service.go	

Description

The dynamic contract call path includes an unconditional time.Sleep(1s). It then calls waitForContractCode, which may poll the user-supplied target contract address up to 10 times with a 2-second delay.

An attacker can repeatedly submit requests pointing to addresses without deployed code. Each request can block a worker or connection for a long period, potentially exhausting server resources under concurrent load.

Recommendation

Remove the unconditional sleep.

Status

This issue has been resolved by the team.

14. Integer Arguments Can Suffer Silent Precision Loss Through float64

Severity: Medium	Category: Numerics
Target: - rest/tokens/service.go	

Description

JSON numbers are commonly decoded into float64 when using generic interface{} values. The convertToInteger function converts float64 directly to int64 and then to big.Int.

IEEE 754 double precision cannot exactly represent integers above 2^53. Large token amounts or integer parameters may be silently rounded, causing the backend to sign or submit on-chain calls with values different from the user's intent.

Recommendation

For ABI integer parameters, accept only decimal strings or explicitly parsed json.Number values.

Reject floating-point values for integer ABI slots. Add unit tests for 2^53 - 1, 2^53, 2^53 + 1, and very large integers to ensure invalid inputs are rejected instead of silently truncated.

Status

This issue has been resolved by the team.

15. Hardcoded Private Keys in Test and Script Files

Severity: Low	Category: Configuration
Target: - scripts/test_gas.js - main/test.js	

Description

Several scripts contain hardcoded private keys. While these may be intended for local testing, committing private keys is an unsafe development practice.

If these accounts are ever funded on a live network, or if the scripts are reused in a production-like environment, the funds can be stolen. Hardcoded keys also increase the risk of accidental disclosure through version control, logs, screenshots, or copied examples.

Recommendation

Move private keys to environment variables or local .env files that are excluded from version control.

Use clearly documented test-only mnemonic accounts for local development. If any committed key was ever used with real funds, rotate it immediately and move all assets to a new address.

Status

This issue has been resolved by the team.

2.3 Informational Findings

16. The parseContractResult Does Not Validate Empty Return Values

Severity: Informational	Category: Business Logic
Target: - rest/tokens/service.go	

Description

In the dynamic read-only call flow, the backend calls parseContractResult after contract.Call succeeds.

For non-tuple return values, the function directly returns result[0] without first checking len(result). If a contract returns an unexpected empty result, the backend may panic or fail with an unhandled runtime error.

Recommendation

Validate len(result) before accessing result[0].

Also compare the number of returned values against the ABI method's expected output count and return a controlled error if the result shape is invalid.

Status

This issue has been resolved by the team.

Appendix

Appendix 1 - Files in Scope

This audit covered the following files in commit :

File	SHA-1 hash
core/ach_gas.go	337591c6c87a0891f3d0444003a6ddba5e49f56b
core/state/statedb.go	17c0990443f3043e04d6ec3eb056b2eae49945af
core/state_transition.go	7ba81edf3ef0fde9937a6456aebd46857766b839
core/types/receipt.go	da61f3a0b4b39a74b486faab83569f1c6eed1408
core/types/tx_dynamic_fee.go	ae541de8244728e1046dbba5cd50f333c28f2b87
core/txpool/validation.go	f488aa797308bd2ad63bb288290f945d2e5f77bf
core/vm/interface.go	9144afe9d3ce19ec35b9135d3db63b9e2fdc437f
cmd/geth/config.go	130c4b6dbc2beae504e7eb102b13b4734e52f469
eth/backend.go	41f6d50ea52875e033158e14eadad720c5d2cc3f
params/config.go	3a22193102e4e82ef0219115a715a975a9a5e805
params/denomination.go	533ffbdbd111332311b1493aa731ddb9dbbe4625
rest/tokens/service.go	ab166f1e635d79f5360bc7d218a405dfc420f064
rest/tokens/crypto/signature.go	a440ea1c6839affb2a1ddd477d9a9845498abd89
rest/tokens/utils/validation.go	c96a2b161a1dbbf2b513daae3ab6a482407f72b3
rest/tokens/security/crypto_utils.go	3e17e72f1a9db536f4c7abc1b84e2f4e9d940e6a
rest/tokens/types/config.go	7db8314dd2e5d5d192aedd3db6d6ed4546767286
rest/tokens/types/request.go	e815666030f54a723151d07a9b8f2a99b34c04b6
rest/tokens/keyutils/generate_keys.go	a23d0b5ca1cc5bb99f51925238ed9106f412b247
rest/tokens/utils/http.go	44d3468c3445eba833e74d4b86398dfdbee3b165
rest/tokens/utils/hex.go	3656515dbda1138b2f4526b13e904e02072d0c07
rest/tokens/types/contract.go	c4867c3ec1564551ef75f41575cc3c2e1edbb2c5
rest/tokens/contracts/ERC20BaseUpgradeable.sol	a67d4f6246f0687a03d1839065fa4bc6202d3fc6
rest/tokens/contracts/Permission.sol	ecd79e629a77210b22fc9028828e76b8ba057f06

rest/tokens/contracts/SimpleERC20.sol	8e99befa5ca53542970e5834e60ad094e3d2a1a4
rest/tokens/contracts/MyERC20Proxy.sol	16a0a8ed0cc432b7d1089af05ae99a1499911265
main/req.go	2018947e8c1291d68717776de48bb38198f8c274
main/decode_logs.go	622a1533c2b88e590fa4c5805c3d518d68155dae
genesis.json	8f887497fc72fa859876802fa9a054c66acacd9f

And this audit covered the following fixed versions of the files:

File	SHA-1 hash
core/ach_gas.go	4bbcf88ab71e0882ebad8c6196be0bd217d51108
core/state/statedb.go	2a91aa9408d482b4deef00c18af56a584c826e25
core/state_transition.go	7ba81edf3ef0fde9937a6456aebd46857766b839
core/types/receipt.go	a13def33d9d195970268255bf44ef866c4a8795b
core/types/tx_dynamic_fee.go	ae541de8244728e1046dbba5cd50f333c28f2b87
core/txpool/validation.go	d64d7a430e3461f496d8994b442e6a8991f01a18
core/vm/interface.go	9144afe9d3ce19ec35b9135d3db63b9e2fdc437f
cmd/geth/config.go	130c4b6dbc2beae504e7eb102b13b4734e52f469
eth/backend.go	41f6d50ea52875e033158e14eadad720c5d2cc3f
params/config.go	3a22193102e4e82ef0219115a715a975a9a5e805
params/denomination.go	533ffbdbd111332311b1493aa731ddb9dbbe4625
rest/tokens/service.go	ac7cbe34015ef533a532e8a8edb477923ff25d5b
rest/tokens/crypto/signature.go	53f6e971dcd3da089dde166fda1cb839511a0206
rest/tokens/utils/validation.go	5bfa66cb20955fe3c7c123cdd6cf10d175d3f6a1
rest/tokens/security/crypto_utils.go	3e17e72f1a9db536f4c7abc1b84e2f4e9d940e6a
rest/tokens/types/config.go	20262caad2a1a703e7b82ccfa930e748798eac13
rest/tokens/types/request.go	d7a11400db79bdd128c6569164945bb971622f84
rest/tokens/keyutils/generate_keys.go	0a6af522c94b57c72f4dc301f220ee24772ef1dd
rest/tokens/utils/http.go	44d3468c3445eba833e74d4b86398dfdbee3b165
rest/tokens/utils/hex.go	3656515dbda1138b2f4526b13e904e02072d0c07
rest/tokens/types/contract.go	c4867c3ec1564551ef75f41575cc3c2e1edbb2c5

rest/tokens/contracts/ERC20BaseUpgradeable.sol	689756050831711282962472059241856c82de77
rest/tokens/contracts/Permission.sol	ecd79e629a77210b22fc9028828e76b8ba057f06
rest/tokens/contracts/SimpleERC20.sol	8e99befa5ca53542970e5834e60ad094e3d2a1a4
rest/tokens/contracts/MyERC20Proxy.sol	16a0a8ed0cc432b7d1089af05ae99a1499911265
main/req.go	2018947e8c1291d68717776de48bb38198f8c274
main/decode_logs.go	622a1533c2b88e590fa4c5805c3d518d68155dae
genesis.json	8ddd8f5eb25399461a424f10c57d3232fe86810d